

Technická univerzita v Košiciach
Fakulta elektrotechniky a informatiky

Formálne jazyky
Dokumentácia k obhajobe zadania

Dmytro Shkyl
Akademický rok: 2024/2025
Študijný program: informatika

Obsah

1	Formulácia zadania	2
2	Vypracovanie zadania	3
2.1	Opis architektúry riešenia	3
2.2	Gramatika jazyka regulárnych výrazov	3
2.3	Podrobný postup implementácie	4
2.4	Ukážka vstupu a prezentácia výsledkov	5
2.5	Problémy a ich riešenie	9
2.6	Hodnotenie zadania	9
3	Vyhodnotenie a záver	9

1 Formulácia zadania

Téma: Generátor konečnostavových automatov

Cieľ zadania: Implementovať generátor nedeterministických konečnostavových automatov (NKA) na základe zadaného regulárneho výrazu. Pri implementácii je potrebné využiť metódu syntaktickej analýzy zhora nadol (rekurzívny zostup).

Požiadavky na vstup:

- Vstupom je reťazec znakov, ktorý reprezentuje regulárny výraz v štandardne využívanej notácii pre predmet Formálne jazyky.

Výstup programu:

- Grafická reprezentácia odvodeného stromu pre zadaný regulárny výraz.
- Plne funkčná programová implementácia nedeterministického konečnostavového automatu zodpovedajúceho zadanému regulárnemu výrazu.

2 Vypracovanie zadania

2.1 Opis architektúry riešenia

Architektúra je navrhnutá modulárne, aby bolo možné jednotlivé časti programu samostatne testovať. Navrhnuté riešenie je rozdelené do štyroch hlavných modulov:

- **Lexer**
Lexer zodpovedá za rozdelenie vstupného reťazca (regulárneho výrazu) na tokeny. Spracováva jednotlivé symboly.
- **Parser**
Parser prijíma tokeny z lexera a pomocou metódy rekurzívneho zostupu vytvára syntaktický strom, ktorý reprezentuje štruktúru regulárneho výrazu.
- **Generátor NKA**
Tento modul vytvára nedeterministický konečnostavový automat (NKA) na základe syntaktického stromu. Každá operácia regulárneho výrazu má definované pravidlá.
- **Main**
Modul `main` zabezpečuje spustenie programu, načítanie vstupného výrazu, inicializáciu lexera, parsera a generátora NKA. Tiež môže zobrazovať grafickú reprezentáciu odvodeného stromu.

2.2 Gramatika jazyka regulárnych výrazov

Gramatika pre regulárne výrazy môže byť definovaná ako:

- **Symbole:**

$$R \rightarrow a \mid b \mid c \mid \dots \quad (\text{symbole ako } a, b, c, \dots)$$

- **Postupnosť:**

$$R \rightarrow RR \quad (\text{sekvencia dvoch symbolov})$$

- **Alternatíva:**

$$R \rightarrow R \mid R$$

- **Pozitívny uzáver:**

$$R \rightarrow R\{R\} \quad (\text{jeden alebo viac opakovaní})$$

- **Tranzitívny uzáver:**

$$R \rightarrow \{R\} \quad (\text{opakovanie nula alebo viackrát})$$

- **Voliteľnosť:**

$$R \rightarrow [R] \quad (\text{nula alebo jedenkrát})$$

2.3 Podrobný postup implementácie

1. Lexikálny analyzátor (Lexer)

Lexikálny analyzátor je zodpovedný za rozdelenie vstupného reťazca na jednotlivé tokeny, ktoré sú základom pre ďalší syntaktický rozbor. Implementácia lexikálneho analyzátora zahŕňa:

- **Inicjalizácia:** Lexer sa inicializuje s reťazcom, ktorý sa má analyzovať. Určuje sa počiatočná pozícia na začiatok reťazca.
- **Metóda `advance()`:** Posúva aktuálnu pozíciu o jeden znak v reťazci a aktualizuje aktuálny znak. Ak sa pozícia dostane za koniec reťazca, nastaví sa aktuálny znak na `None`.
- **Metóda `skip_whitespace()`:** Preskakuje všetky medzery a nové riadky, aby analyzátor mohol pokračovať v spracovaní tokenov.
- **Metóda `get_next_token()`:** Hlavná metóda lexikálneho analyzátora, ktorá prechádza vstupný reťazec a vracia nasledujúci token podľa rozpoznaných symbolov.

Výsledkom lexikálneho analyzátora sú tokeny, ako napríklad symboly (písmená), operátory, skratky pre zátvorky a iné špeciálne znaky.

2. Syntaktický analyzátor (Parser)

Syntaktický analyzátor spracováva sekvenciu tokenov v syntaktické stromy, ktoré reprezentujú štruktúru pravidiel pre dané regulárne výrazy. Implementácia je založená na algoritme rekursívneho spúšťania, ktorý sa skladá z niekoľkých metód pre analýzu rôznych častí regulárnych výrazov.

- **Metóda `Alternative()`:** Táto metóda analyzuje alternatívy (výrazy spojené operátorom `|`), ale ak sa v rámci regulárneho výrazu nenájde operátor `|`, vráti sekvenciu. Rekurzívne volá metódu `Sequence()` na analyzovanie jednotlivých alternatív. Ak sa zistí `|`, vytvorí sa nový uzol v strome pre alternatívu.

- **Metóda `Sequence()`**: Analyzuje sekvencie, kde výrazy musia byť spracované v presnom poradí. Umožňuje kombinovať viaceré faktory do sekvencie.
- **Metóda `Factor()`**: Analyzuje jednotlivé faktory, ako sú symboly, zátvorky. Na základe tokenu rozhodne, ako tento token spracovať.

Pri spracovaní regulárnych výrazov sa používajú rôzne typy uzlov, ktoré reprezentujú elementy regulárnych výrazov. Tieto uzly sú:

- **regular**: reprezentuje regulárny výraz.
- **element**: reprezentuje symboly.
- **alternative**: reprezentuje alternatívu (výrazy oddelené operátorom `|`).
- **sequence**: reprezentuje sekvenciu (výrazy vykonané v poradí).
- **LCBRA a RCBRA**: reprezentuje opakovanie (výraz v uzavretých zátvorkách `{}`).
- **LBRCKT a RBRCKT**: reprezentuje voliteľnosť (výraz v hranatých zátvorkách `[]`).

3. Vytváranie a vizualizácia stromu

Na vizualizáciu syntaktického stromu sa použila knižnica **anytree**. Vizualizácia umožňuje zobrazovať štruktúru regulárneho výrazu v podobe stromu s uzlami reprezentujúcimi jednotlivé časti výrazu. Tento proces zahŕňa:

- **Metóda `has_pipe()`**: Prechádza strom a zisťuje, či existuje operátor `|`. To je dôležité pre správne vykreslenie alternatívnych výrazu.
- **Metóda `build_tree()`**: Rekurzívne konštruuje strom, pričom pre každý uzol vyhodnocuje jeho typ a pridáva príslušné deti do stromu.

V prípade, že strom obsahuje operátor `|`, vytvorí sa nový uzol typu **alternative**, ktorý je potom napojený na ďalšie uzly.

2.4 Ukážka vstupu a prezentácia výsledkov

Pre spracovanie regulárnych výrazov sa používa rôzne výrazy, ktoré sú spracované na syntaktické stromy. Príklad vstupu:

- $\{ab|c\}$: Tento regulárny výraz sa používa na opakovanie reťazcov, ktoré začínajú písmenom **a**, následne obsahujú **b** alebo **c**. Prazdny reťazec tiež bude akceptovaný.

```

regular
- alternative
  - sequence
    - element
      - <LCBRA>
      - regular
        - alternative
          - sequence
            - element
              - SYMBOL
              - a
            - element
              - SYMBOL
              - b
          - PIPE
          - sequence
            - element
              - SYMBOL
              - c
        - <RCBRA>

```

Pre spracovanie regulárneho výrazu $\{ab|c\}$ môžeme generovať konečný automat (NKA), ktorý akceptuje nasledovné reťazce:

1. **ab**: Akceptované
2. **c**: Akceptované
3. **a**: Neakceptované
4. **abc**: Akceptované
5. (prázdný reťazec): Akceptované

- [a] [b] [c]: Tento regulárny výraz znamená, že každé písmeno môže prísť nula alebo jeden krát. Prazdny retazec tiež bude akceptovaný.

```

regular
- sequence
| - element
|   - <LCBRCKT>
|   - regular
|     - element
|       - SYMBOL
|         - a
|   - <RCBRCKT>
| - element
|   - <LCBRCKT>
|   - regular
|     - element
|       - SYMBOL
|         - b
|   - <RCBRCKT>
| - element
|   - <LCBRCKT>
|   - regular
|     - element
|       - SYMBOL
|         - c
|   - <RCBRCKT>

```

Pre spracovanie regulárneho výrazu [a] [b] [c] môžeme generovať konečný automat (NKA), ktorý akceptuje nasledovné reťazce:

1. ab: Akceptované
2. c: Akceptované
3. abc: Akceptované
4. ca: Neakceptované
5. bac: Neakceptované
6. (prázdny reťazec): Akceptované

- `abcdef`: Tento regulárny výraz znamená, že symboly musia prichádzať postupne. Prázdny reťazec nebude akceptovaný.

```
regular
- sequence
| -element
|   -SYMBOL
|     - a
| -element
|   -SYMBOL
|     - b
| -element
|   -SYMBOL
|     - c
| -element
|   -SYMBOL
|     - d
| -element
|   -SYMBOL
|     - e
| -element
|   -SYMBOL
|     - f
```

Pre spracovanie regulárneho výrazu `abcdef` môžeme generovať konečný automat (NKA), ktorý akceptuje nasledovné reťazce:

1. `ab`: Neakceptované
2. `c`: Neakceptované
3. `a`: Neakceptované
4. `abcdef`: Akceptované
5. (prázdny reťazec): Neakceptované

2.5 Problémy a ich riešenie

Počas implementácie som sa stretol s problémom, že pri použití zátvoriek ($\{\}$), ktoré označujú opakovanie, sa môj vstup neopakoval a slovo nebolo akceptované správne.

Aby som tento problém vyriešil, vytvoril som boolean premennú `has_LCBRA`, ktorá označuje, že v aktuálnom výraze sa nachádza ľavá zátvorka. Táto premenná umožňuje rozpoznať, kedy sa nachádzame vnútri zátvoriek, a preto sme po jej identifikovaní vygenerovali epsilonový prechod, ktorý umožnil "preskočiť" späť na štartovací stav. Týmto spôsobom sa zabezpečilo, že výraz s opakovaním fungoval správne a akceptoval slová, ktoré by sa inak nezrealizovali.

2.6 Hodnotenie zadania

V celkovom kontexte bolo zadanie veľmi zaujímavé, pretože si vyžiadalo podrobné porozumenie konceptu práce NKA. Aby sme správne implementovali generátor NKA na základe regulárnych výrazov, bolo potrebné dôkladne pochopiť, ako NKA funguje a ako môžeme využiť jeho vlastnosti na správne spracovanie rôznych komponentov regulárnych výrazov, ako sú sekvencie, alternatívy a opakovania.

3 Vyhodnotenie a záver

Počas implementácie sa mi podarilo správne vybudovať štruktúru stromu a implementovať základné komponenty regulérnych výrazov, ako sú sekvencie, alternatívy a opakovanie symbolov.

Okrem toho som implementoval správnu logiku pre spracovanie epsilonových prechodov v prípadoch s opakovaním, čo umožnilo správnu interpretáciu regulérnych výrazov s opakujúcimi sa vzormi.

Aj keď úloha bola náročná, považujem ju za veľmi poučnú, pretože poskytnúla príležitosť na hlbšie pochopenie koncepcie regulérnych výrazov a ich implementácie v podobe NKA.